

1. Consider a modification to the activity-selection problem in which each activity a_i has, in addition to a start and finish time, a value v_i . The objective is no longer to maximize the number of activities scheduled, but instead to maximize the total value of the activities scheduled. That is, we wish to choose a set A of compatible activities such that $\sum_{a_k \in A} v_k$ is maximized.

i	1	2	3	4	5	6	7
s_i	3	2	5	1	8	7	10
f_i	7	5	9	4	9	9	12
v_i	3	4	2	3	1	2	3

- Determine an optimal solution for the given table. 根據上表、找出最佳解
- Give a polynomial-time algorithm for this problem.

1. Algorithm

$dp[i]$: 活動 $1 \sim i$ 的最佳值

對完成時間擇升序排列

$$dp[0] = 0$$

for $i = 1$ to n ($n = 7$)

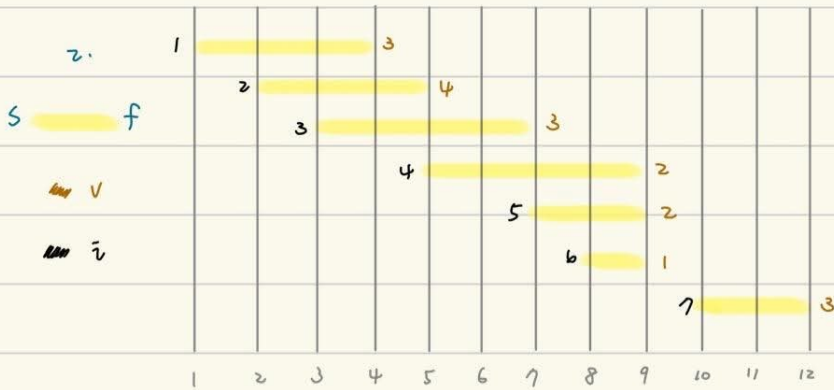
$last =$ 以二分搜尋法找完成時間 $\leq s_i$ 的活動 $j \rightarrow O(\lg n)$

$$dp[i] = \max \left(\underset{\substack{\downarrow \\ \text{目前活動不選}}}{dp[i-1]}, \underset{\substack{\downarrow \\ \text{選目前活動}}}{dp[last] + v_i} \right)$$

最後Ans為 $dp[n]$

$$\text{time complexity} = n \times O(\lg n) = O(n \lg n)$$

i	1	2	3	4	5	6	7
s_i	3	2	5	1	8	7	10
f_i	7	5	9	4	9	9	12
v_i	3	4	2	3	1	2	3



dp 1 2 3 4 5 6 7

max (0, 0+3) (3, 0+4) (4, 0+3) (4, 4+2) (6, 4+2) (6, 4+1) (6, 6+3)

3 4 4 6 6 6 9

i	1	2	3	4	5	6	7
s_i	3	2	5	1	8	7	10
f_i	7	5	9	4	9	9	12
v_i	3	4	2	3	1	2	3

→ 不能对活動價值進行排序，會導致一個價值高但長的活动蓋住2個低價但合起來價值更高的短活動！
要對結束時間進行排序！！

$$\text{轉移式: } dp[i] = \max(dp[i-1], dp[p[i]] + v_i)$$

不選現在的活動 ← 選現在的活動，去找選現在這個還能配哪些活動

如果決定選活動 i ，就需要往前找 → 在活動 i 前能搭配哪些活動？

⚡ 先前已經按結束時間排序並重新編號！

$p(i)$ → 排序後編號 $< i$ 的活動中，最後一個能和活動 i 相容者。

編號最大者 → 結束時間最晚者

*換言之，活動 $1 \sim p(i)$ 都有可能和活動 i 共存，而保證活動 $p(i)+1 \sim i-1$ 必和活動 i 衝突

→ 而直接選擇結束時間最晚者能讓轉移的性質變乾淨。

若 $f_j \leq s_i$ 成立，那對於所有 $j' < j$ 也都有 $f_{j'} \leq f_j \leq s_i$ ，保證在當前轉移選了活動 i 後，子問題為前 $p(i)$ 個活動的最佳解。!!

$p[i] = \max_{0 \leq j < i} \{f_j \leq s_i \text{ 者}\}$ ，若不存在任何 $j < i$ 滿足 $f_j \leq s_i$ 者，即表沒有任何活動相容，填 0

→ 因為活動結束時間已排序，所以可以二分搜來找最大的 j 使得 $f_j \leq s_i \rightarrow n \times O(\lg n)$

2,

i	1	2	3	4	5	6	7
s _i	3	2	5	1	8	7	10
f _i	7	5	9	4	9	9	12
v _i	3	4	2	3	1	2	3

→ 不能对活動價值進行排序, 會導致一個價值高但長的活动蓋住 2 个低價但合起來價值更高的短活動!

要对結束時間進行排序!!

轉移式: $dp[i] = \max(dp[i-1], dp[p[i]] + v_i)$

不選現在的活動 ← 選現在的活動, 去找選現在這個還能配哪些活動

Activity Selection With Value (activities, n)

create an array $dp[0 \sim n]$

sort activities by finish time, 由小到大, 並重新編號 → $O(n \lg n)$

create an array $p[1 \sim n]$

利用二分查找最大的 j 使得 $f_j \leq s_i$, 並將結果存入 $p \rightarrow n \times O(\lg n) = O(n \lg n)$

$dp[0] = 0$

for $i \leftarrow 1 \sim n$:

$dp[i] = \max(dp[i-1], dp[p[i]] + \text{activities}[i].v) \rightarrow O(n)$

return $dp[n]$

→ Time Complexity: $O(n \lg n)$

Creating p by binary Search:

for $i \leftarrow 1 \sim n$:

$l \leftarrow 1, r \leftarrow i-1, p[i] \leftarrow 0$

while $l \leq r$:

$mid = (l+r)/2$

if $\text{activities}[mid].f \leq \text{activities}[i].s$

$p[i] = mid$

$l = mid + 1$

else

$r = mid - 1$

1、

i	1	2	3	4	5	6	7
s_i	3	2	5	1	8	7	10
f_i	7	5	9	4	9	9	12
v_i	3	4	2	3	1	2	3

轉移式: $dp[i] = \max(dp[i-1], dp[p[i]] + v_i)$

不選現在的活動 $\leftarrow$ $\rightarrow$ 選現在的活動 $\rightarrow$ 去找選現在這個還能配哪些活動

i	1	2	3	4	5	6	7
s_i	1	2	3	5	8	7	10
f_i	4	5	7	9	9	9	12
v_i	3	4	3	2	1	2	3
P_i	0	0	0	2	3	3	6

$P(1)$: 前面沒有任何活動 $\rightarrow P(1) = 0$

$P(2)$: $S_2 = 2$, 向前找 $f_j \leq 2 \rightarrow$ 不存在, $P(2) = 0$

$P(3)$: $S_3 = 3$, 向前找 $f_j \leq 3 \rightarrow$ 不存在, $P(3) = 0$

$P(4)$: $S_4 = 5$, 向前找 $f_j \leq 5 \rightarrow$ 存在 $f_2 = 5$, $P(4) = 2$

$P(5)$: $S_5 = 8$, 向前找 $f_j \leq 8 \rightarrow$ 存在 $f_3 = 7$, $P(5) = 3$

$P(6)$: $S_6 = 7$, 向前找 $f_j \leq 7 \rightarrow$ 存在 $f_3 = 7$, $P(6) = 3$

$P(7)$: $S_7 = 10$, 向前找 $f_j \leq 10 \rightarrow$ 存在 $f_6 = 9$, $P(7) = 6$

	0	1	2	3	4	5	6	7
dp	0	3	4	4	6	6	6	9

$$dp[1] = \max(\underbrace{dp[0]}_0, \underbrace{dp[p[1]] + 3}_{0+3=3})$$

$$dp[2] = \max(\underbrace{dp[1]}_3, \underbrace{dp[p[2]] + 4}_{0+4=4})$$

$$dp[3] = \max(\underbrace{dp[2]}_4, \underbrace{dp[p[3]] + 3}_{0+3=3})$$

$$dp[4] = \max(\underbrace{dp[3]}_4, \underbrace{dp[p[4]] + 2}_{4+2=6})$$

$$dp[5] = \max(\underbrace{dp[4]}_6, \underbrace{dp[p[5]] + 1}_{3+1=4})$$

$$dp[6] = \max(\underbrace{dp[5]}_6, \underbrace{dp[p[6]] + 2}_{4+2=6})$$

$$dp[7] = \max(\underbrace{dp[6]}_6, \underbrace{dp[p[7]] + 3}_{6+3=9})$$

1. Minimizing lateness problem.

- Single resource processes one job at a time.
- Job i requires t_i units of processing time and is due at time d_i .
- If i starts at time s_i , it finishes at time $f_i = s_i + t_i$.
- Lateness: $\ell_i = \max\{0, f_i - d_i\}$.
- Goal: schedule all jobs to minimize maximum lateness $L = \max\{\ell_i \mid 1 \leq i \leq n\}$.

i	1	2	3	4	5	6	7
t_i	3	3	5	1	2	3	2
d_i	7	8	9	8	9	14	16

(1) 貪婪演算法, d_i 愈早, i 項愈早排

i	1	2	3	4	5	6	7
t_i	3	3	5	1	2	3	2
s_i	0	3	7	6	12	14	17
f_i	3	6	12	7	14	17	19
d_i	7 _①	8 _②	9 _④	8 _③	9 _⑤	14 _⑥	16 _⑦
ℓ_i	0	0	3	0	5	3	3

5分 $\rightarrow L = \max\{0, 3, 5\} = 5$

($\geq$) d_i 由愈早, job 愈早執行

Algorithm:

① d_i 由小排到大, t_i 根據 d_i 結果一起排

② $s[1] = 0, f[1] = 0$

③ for 1 to N ↗ assume there are N jobs.

$$s[i] = f[i]$$

$$f[i] = s[i] + t[i]$$

$$l[i] = \max \{ 0, f[i] - d[i] \}$$

④ $L = \max \{ l, l + n \}$

5分

如何
排程

找
lateness

1. Minimizing lateness problem.

- Single resource processes one job at a time.
- Job i requires t_i units of processing time and is due at time d_i .
- If i starts at time s_i , it finishes at time $f_i = s_i + t_i$.
- Lateness: $\ell_i = \max\{0, f_i - d_i\}$.
- Goal: schedule all jobs to minimize maximum lateness $L = \max\{\ell_i \mid 1 \leq i \leq n\}$.

最小化最大延遲

i	1	2	3	4	5	6	7
t_i	3	3	5	1	2	3	2
d_i	7	8	9	8	9	14	16

1. 根據此表找最佳解

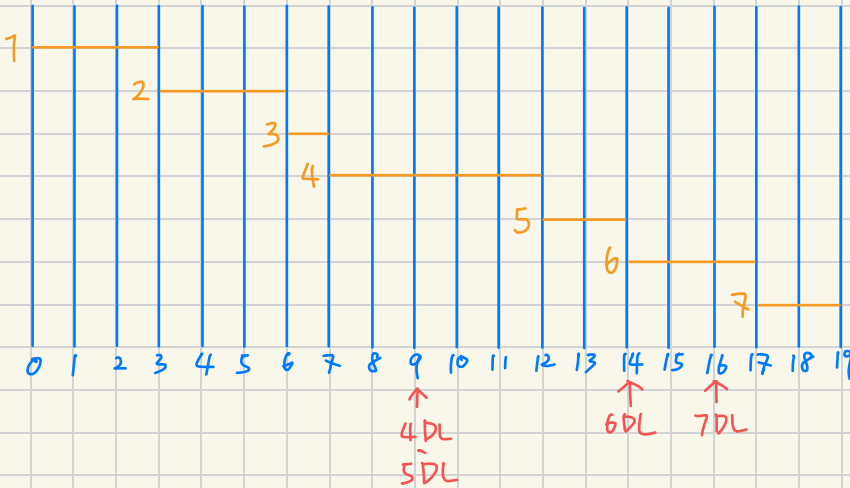
2. 设计一多项式時間 Algorithm to solve problem.

1. Greedy Algorithm $\rightarrow$ 截止時間(d_i) 越早越先做!

Sort the job by d_i and re-labeling the job

i	1	2	3	4	5	6	7
t_i	3	3	1	5	2	3	2
d_i	7	8	8	9	9	14	16
s_i	0	3	6	7	12	14	17
f_i	3	6	7	12	14	17	19
ℓ_i	0	0	0	3	5	3	3

$$L = \max(\{0, 3, 5\}) = 5$$



1. Minimizing lateness problem.

- Single resource processes one job at a time.
- Job i requires t_i units of processing time and is due at time d_i .
- If i starts at time s_i , it finishes at time $f_i = s_i + t_i$.
- Lateness: $\ell_i = \max\{0, f_i - d_i\}$.
- Goal: schedule all jobs to minimize maximum lateness $L = \max\{\ell_i \mid 1 \leq i \leq n\}$.

i	1	2	3	4	5	6	7
t_i	3	3	5	1	2	3	2
d_i	7	8	9	8	9	14	16

最小化最大延遲

1. 根據此表找最佳解

2. 设计一多项式時間 Algorithm to solve problem.

2.

Assume exist job $1 \sim n$

MiniLate (jobs, n)

sort jobs by d_i 由小到大, 並重新編號 $\rightarrow O(n \lg n)$

jobs[0].s = 0, jobs[0].f = 0

$L = 0$

for $i \leftarrow 1 \sim N$:

jobs[i].s = jobs[i-1].f

jobs[i].f = jobs[i].s + jobs[i].t

nowlate = jobs[i].f - jobs[i].d

if nowlate > 0:

$L = \max(\text{nowlate}, L)$;

return L ;

$\rightarrow O(n)$

$\rightarrow$ Time Complexity = $O(n \lg n) + O(n) = O(n \lg n)$